



RYS

Restoran Yönetim Sistemi

Final Proje Raporu

Üniversite	Necmettin Erbakan Üniversitesi
Bölüm	Bilgisayar Mühendisliği
Ders	BİL 204 — Yazılım Mühendisliği
Grup	Grup 8
Ekip Üyeleri	Ramazan Faruk Acıöz/Mehmet Selim Ceran/Muhammed Yusuf Uğur/Serdar Kara/Hüseyin Kaya/İssa Abakar Mahamat
GitHub	https://github.com/rmznfaruk/rys-bil204
Tarih	Mayıs 2026

İçindekiler

1.	Giriş	3
2.	Proje Deneyimi	4
2.1	Analiz Aşaması	4
2.2	Tasarım Aşaması	4
2.3	İmplementasyon Aşaması	5
3.	Yapay Zekâ Kullanımı	6
4.	Kullanıcı Kılavuzu	7
4.1	Giriş / Kayıt Ekranları	7
4.2	Yönetim Paneli	8
4.3	Kullanıcı Yönetimi	9
4.4	Menü Yönetimi	10
4.5	Stok Takip	10
4.6	Masa Planı	11
4.7	Rezervasyon	11
4.8	Sipariş Girişi	12
4.9	KDS — Mutfak Ekranı	12
4.10	Raporlar	13
5.	Kurulum Talimatları	14
5.1	Gereksinimler	14
5.2	Backend Kurulumu	14
5.3	Frontend Kurulumu	15
5.4	Veritabanı Kurulumu	15
6.	Görev Dağılımı	16
7.	Yaptıklarımız, Yapmadıklarımız ve Nedenleri	18

1. Giriş

Restoran Yönetim Sistemi (RYS), BİL 204 — Bilgisayar Mühendisliği dersi kapsamında altı kişilik bir ekip tarafından geliştirilen, restoran operasyonlarını tek platform üzerinden yönetmeye olanak tanıyan full-stack bir web uygulamasıdır.

Proje; **React** tabanlı modern bir kullanıcı arayüzü, **Node.js/Express** REST API sunucusu ve **PostgreSQL** ilişkisel veritabanından oluşmaktadır. Kullanıcı kimlik doğrulama (JWT + BCrypt), masa planı, sipariş akışı, KDS mutfak ekranı, menü/stok yönetimi ve rezervasyon modülleri başarıyla hayata geçirilmiştir.

Sistemin GitHub adresi: <https://github.com/rmznfaruk/rys-bil204>

Teknoloji Katmanı	Araç / Kütüphane	Versiyon
Frontend	React.js	18.x
Backend	Node.js + Express	18.x / 4.x
Veritabanı	PostgreSQL	14+
Kimlik Doğrulama	JWT + BCrypt	—
Geliştirme Ortamı	VS Code, Postman	—

2. Proje Deneyimi

Bu bölümde analiz, tasarım ve implementasyon aşamalarında edindiğimiz deneyimler aktarılmaktadır.

2.1 Analiz Aşaması

Gereksinim analizi sürecinde beş farklı UML diyagramı çizdik: **Use-Case**, **Sınıf (Class)**, **Sıralama (Sequence)**, **Aktivite (Activity)** ve **Durum (State)** diyagramları. Bu süreçte öğrendiklerimiz:

- Fonksiyonel ve fonksiyonel olmayan gereksinimleri birbirinden ayırt etmek
- Use-Case diyagramlarında iç/dış aktörleri ve sistem sınırlarını doğru belirlemek
- Sınıf diyagramlarında MVC ve Entity-Boundary-Control örüntülerini uygulamak
- Aktivite diyagramlarıyla kullanım senaryolarının akışını modellemek
- Durum diyagramlarıyla nesnelerin yaşam döngüsünü görselleştirmek

2.2 Tasarım Aşaması

Tasarım raporunda altsistem ayrıştırması (subsystem decomposition) yaptık. Mimari kararlarımız:

- 3 Katmanlı Mimari:** React SPA (Sunum) → Express REST API (İş Mantığı) → PostgreSQL (Veri)
- JWT Tabanlı Oturum:** Sunucu taraflı session yerine stateless token yönetimi tercih edildi — ölçeklenebilirlik ve mobil uyumluluk için
- Rol Tabanlı Erişim (RBAC):** Yönetici, Garson, Kasiyer, Mutfak rolleri middleware katmanında kontrol edildi
- RESTful API Tasarımı:** Endpoint'ler kaynak odaklı (/api/masalar, /api/siparisler) tasarlandı
- PostgreSQL Tercih Sebebi:** İlişkisel veri yapısı (masa-sipariş-kullanıcı) ve ACID uyumluluğu için MySQL'e kıyasla daha uygun bulundu

2.3 Implementasyon Aşaması

Implementasyon sürecinde her ekip üyesi farklı modülleri üstlenerek paralel geliştirme yaptı. Bu aşamada öğrendiklerimiz:

- React'te bileşen ağacı yönetimi, hook'lar (useState, useEffect, useContext) ve SPA yönlendirme
- Express.js'de middleware zinciri, hata yönetimi ve async/await kullanımı
- PostgreSQL'de JOIN sorguları, indexleme ve bağlantı havuzu (connection pool) optimizasyonu
- BCrypt ile güvenli şifre hashleme ve JWT token ömrü yönetimi

- Git ile paralel dal (branch) stratejisi; feature branch → PR → merge akışı
- API entegrasyonunda CORS ayarı ve frontend–backend bağlantı sorunlarının giderilmesi

Bu sürecin en büyük çıkarımı şu oldu: **Yazılım mühendisliği yalnızca kod yazmak değil; doğru kararları doğru zamanda almak, takım içi iletişimi yönetmek ve belirsizlik altında önceliklendirme yapabilmektir.** Planlama eksikliğinin implementasyon aşamasında katlanarak büyüyen hatalara yol açtığını bizzat deneyimledik.

3. Yapay Zekâ Kullanımı

Geliştirme sürecinde yapay zekâ araçlarını aktif biçimde kullandık. Bu bölüm tamamen şeffaflık amacıyla yazılmıştır.

3.1 Kullanılan Araçlar

Araç	Kullanım Alanı	Kullanım Yoğunluğu
Claude (Anthropic)	Kod inceleme, hata ayıklama, rapor taslakları, UML tasarım gözden geçirme	Yüksek
ChatGPT-4	SQL sorgu optimizasyonu, Express middleware yapılandırması	Orta
GitHub Copilot	Kod tamamlama (boilerplate azaltma), tekrarlayan bileşen üretimi	Orta

3.2 Ne İçin Kullandık?

Hata Ayıklama

JWT middleware'inde 'Cannot read properties of undefined (reading userId)' hatasını ayıklarken Claude'a hata mesajını ve ilgili kod bloğunu gösterdik. Yapay zekâ token payload yapısındaki isimlendirme tutarsızlığını (id vs userId) tespit etti. Önerilen düzeltmeyi anladık, kendi kod yapımıza uyarladık.

React Bileşen Yapısı

Sipariş girişi ekranında kategori-ürün-sepet arasındaki state akışını tasarlarken Copilot ile useState ve prop drilling sorununu tartıştık. Öneri: useContext + useReducer. Bu öneriden yararlanarak kendi implementasyonumuzu yazdık.

Veritabanı Sorguları

Sipariş listesini masa ve kullanıcı bilgileriyle birlikte çekecek JOIN sorgusunda ChatGPT yardım aldık. Üretilen sorguyu test ortamımızda denedik, performans sorununu fark ederek indeks ekledik — bu kararı yapay zekânın önerisi değil, Postman yanıt süreleri ortaya koydu.

UML Diyagramı Gözden Geçirme

Çizdiğimiz sınıf diyagramlarını Claude'a göstererek eksik ilişkileri sorguladık. Sefer rezervasyon ve sipariş sınıfları arasındaki ilişkinin multiplicity'sini (1'e-çok / çok'a-çok) netleştirmemize yardımcı oldu.

Rapor Taslakları

Analiz ve tasarım raporlarının bazı bölümleri için Claude'dan taslak istendi. Üretilen içerik doğrudan kullanılmadı; ekip tartışması sonrası büyük ölçüde yeniden yazıldı.

3.3 Yapay Zekânın Sınırlı Kaldığı Durumlar

- **PostgreSQL bağlantı havuzu ayarı:** Claude başlangıçta yanlış pool konfigürasyonu önerdi (max bağlantı sayısını gereğinden yüksek). Gerçek sunucu testinde bağlantı hatası alındı; biz manuel ayarlayarak düzelttik.
- **CORS yapılandırması:** Copilot'un önerdiği CORS ayarı geliştirme ortamında çalıştı fakat üretim ortamı (farklı port/domain) için yetersizdi. Farkı anlayıp kendiniz çözdük.
- **Proje özgün kararları:** 'Hangi rolün hangi endpoint'e erişebileceği' gibi iş kuralı kararlarını tamamen ekip olarak aldık; AI bu konuda proje bağlamından habersiz olduğu için net öneri sunamadı.

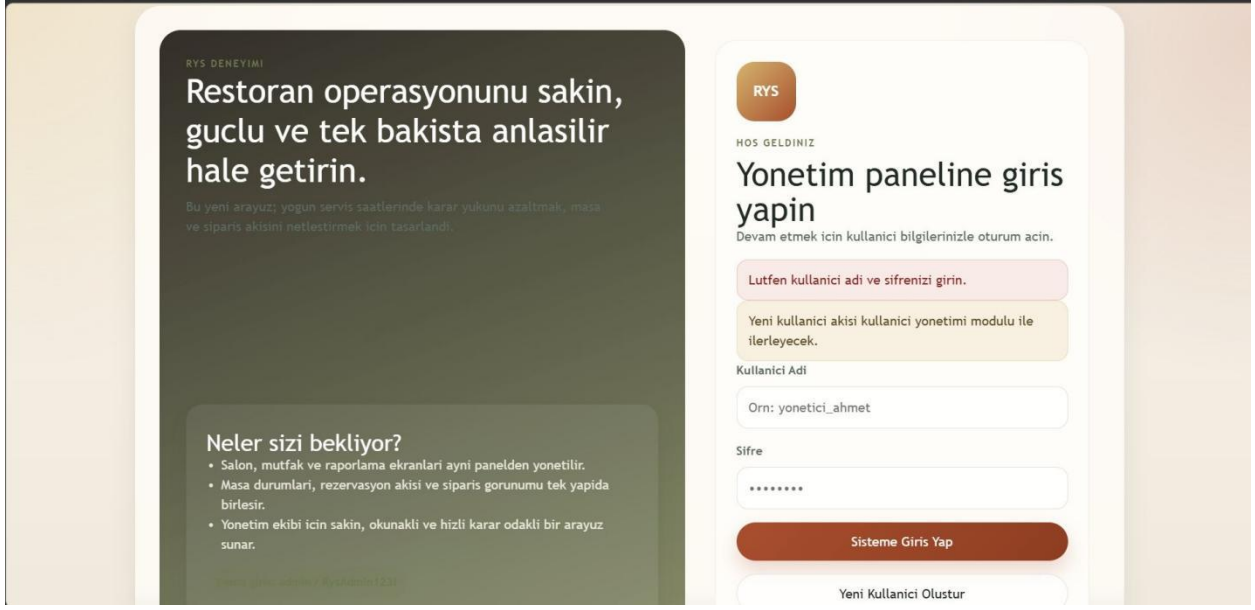
Özet: Yapay zekâ araçları geliştirme sürecimizi hızlandırdı; ancak her öneri eleştirel gözle değerlendirildi. Kodu kopyala-yapıştır yapmak yerine anlamaya odaklandık. AI'nın yanlış yönlendirdiği durumları fark edebildiğimiz için bu yaklaşımın değerli olduğuna inanıyoruz.

4. Kullanıcı Kılavuzu

Aşağıda sistemdeki tüm ekranlar aktör bazında ve sırayla açıklanmaktadır. Her ekran için ilgili ekran görüntüsü eklenmiştir.

4.1 Giriş / Kayıt Ekranları

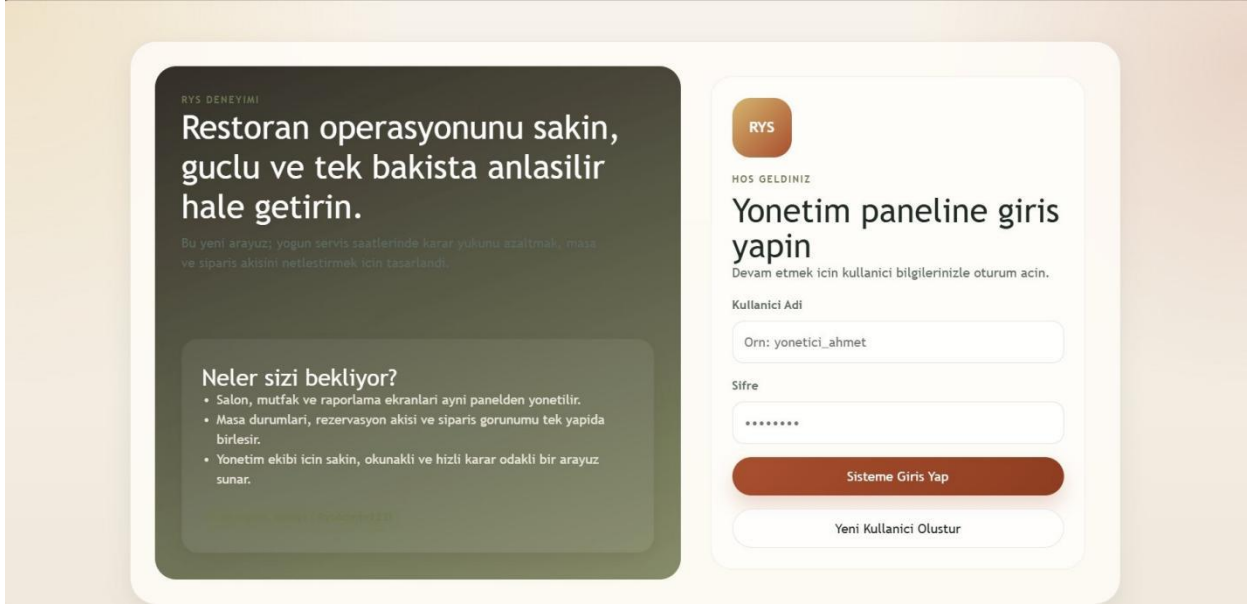
Uygulama tarayıcıda **http://localhost:3000** adresi ile açılır. İlk açılışta sistem React uygulamasını yükler (kısa beyaz ekran görünebilir).



Şekil 1: İlk yüklenme sırasındaki boş ekran (normal davranış)

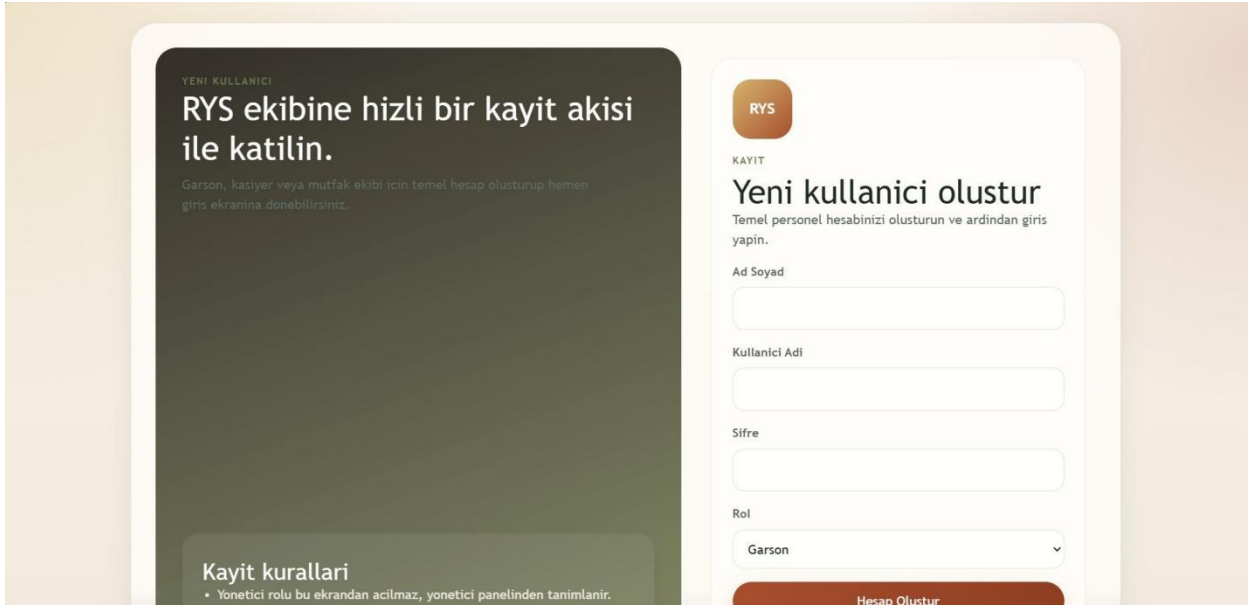
Yükleme tamamlandığında giriş ekranı açılır. Sol tarafta RYS'nin temel özellikleri tanıtılır; sağ tarafta kullanıcı adı ve şifre alanları yer alır.

- **Kullanıcı Adı:** Örnek: yönetici_ahmet (boşluk içermez)
- **Şifre:** Girilen karakterler gizlenir
- **Sisteme Giriş Yap:** Doğrulama başarılıysa Yönetim Paneli'ne yönlendirilirsiniz
- **Hata Durumu:** Yanlış bilgi girildiğinde kırmızı uyarı mesajı görünür



Şekil 2: Giriş ekranı — kullanıcı adı ve şifre alanları

'Yeni Kullanıcı Oluştur' bağlantısına tıklanarak kayıt ekranına geçilir. Ad Soyad, Kullanıcı Adı, Şifre ve Rol (Garson / Kasiyer / Mutfak) doldurulup 'Hesap Oluştur' butonuna basılır.

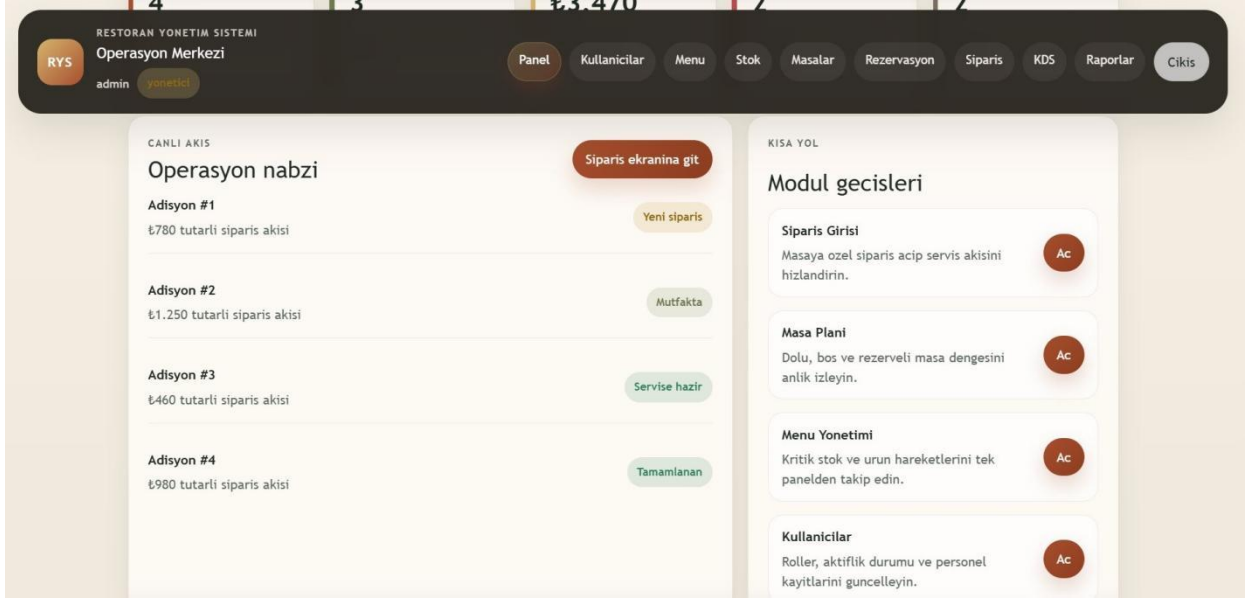


Şekil 3: Kayıt ekranı — yeni personel hesabı oluşturma

4.2 Yönetim Paneli

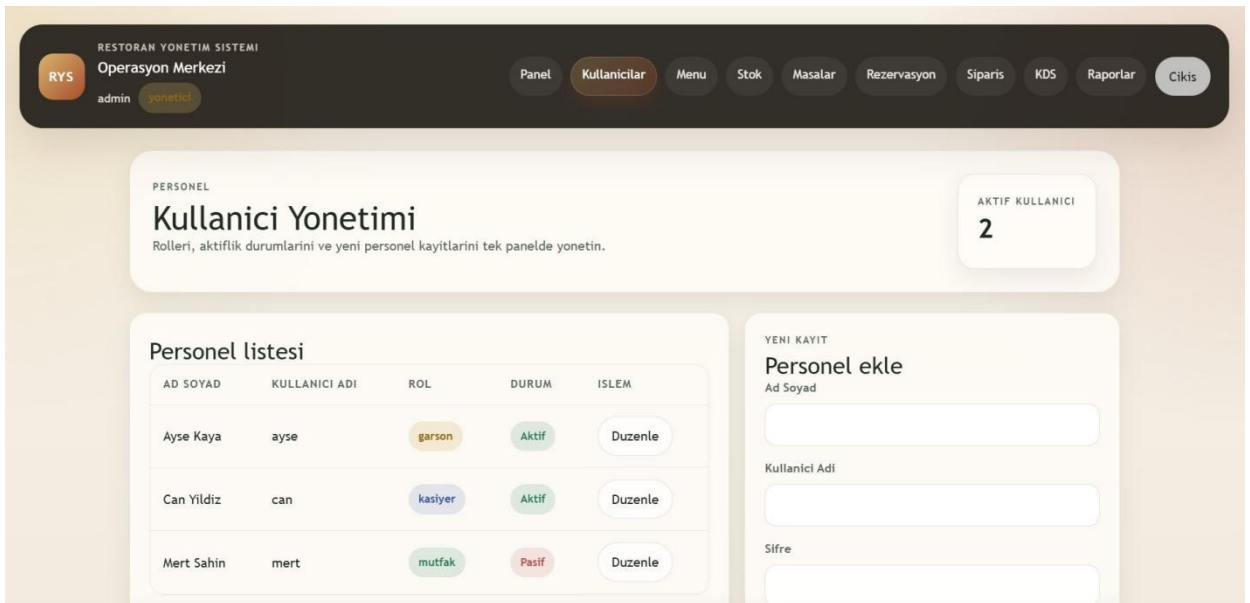
Giriş sonrası açılan ilk ekrandır. Günlük operasyon özetini ve canlı sipariş akışını içerir.

- **Toplam Ciro:** Bugün kapanan siparişlerden elde edilen gelir
- **Aktif Masalar:** Serviste olan ya da rezerveli masa sayısı
- **Kritik Stok:** Acil takip gerektiren ürün sayısı
- **Mutfak Sırası:** Hazırlanmayı bekleyen sipariş adedi



Şekil 4: Yönetim Paneli — üst kısım, özet metrikler

Alt kısımda **Operasyon Nabzı** (canlı adisyon listesi) ve **Modül Geçişleri** (hızlı kısayollar) yer alır.

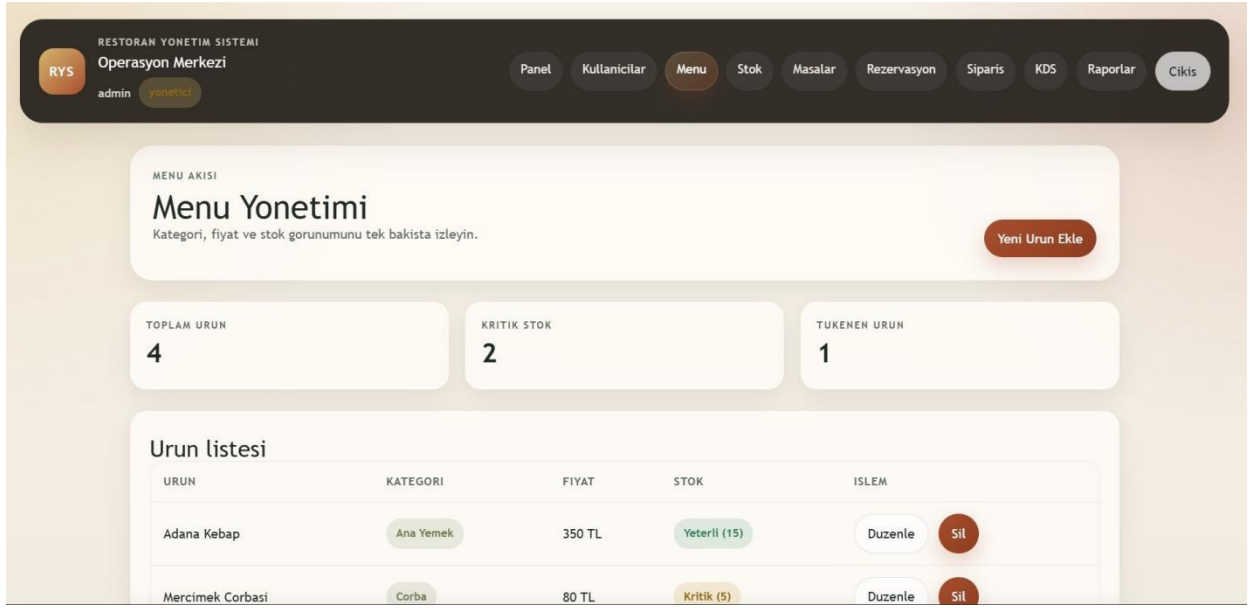


Şekil 5: Yönetim Paneli — Operasyon Nabzı ve Modül Geçişleri

4.3 Kullanıcı Yönetimi

Navbarda 'Kullanıcılar' linkine tıklayın. Ekran iki bölümden oluşur: sol taraf personel listesi, sağ taraf yeni kayıt formu.

- Personel listesinde Ad Soyad, Kullanıcı Adı, Rol (renk rozetli) ve Durum (Aktif/Pasif) görünür
- 'Düzenle' butonu ile mevcut personelin bilgileri güncellenebilir
- Sağdaki form ile yeni personel anlık olarak sisteme eklenebilir
- Üst sağdaki sayaç aktif kullanıcı adedini gösterir

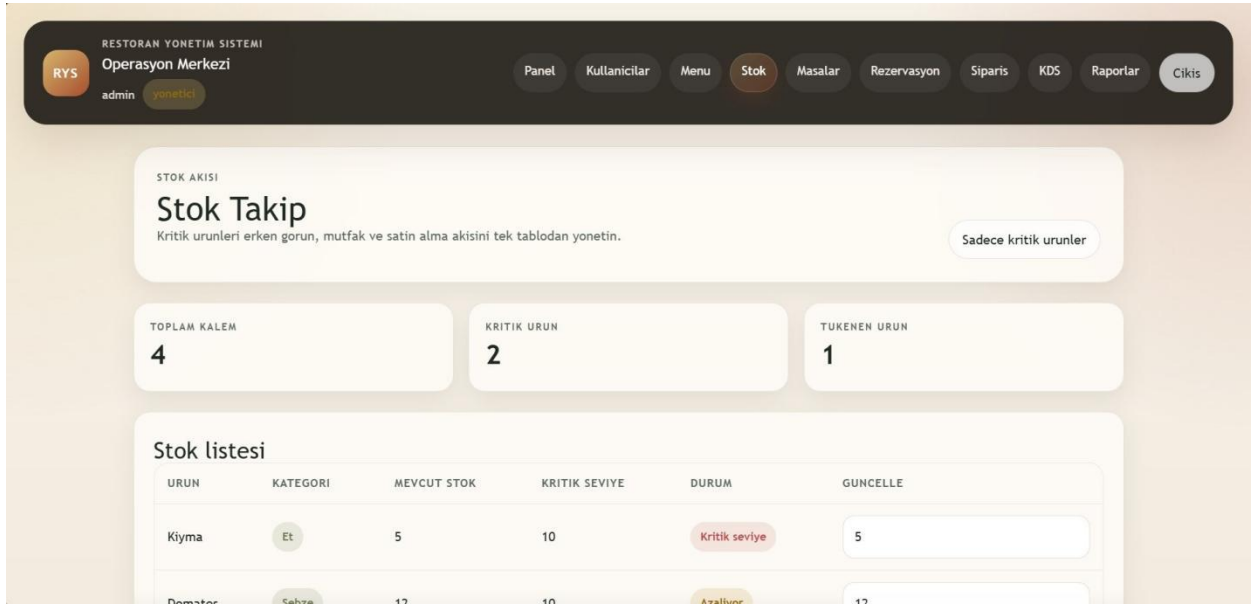


Şekil 6: Kullanıcı Yönetimi — personel listesi ve ekleme formu

4.4 Menü Yönetimi

Navbarda 'Menu' linkine tıklayın. Tüm ürünler kategori, fiyat ve stok durumuyla listelenir.

- **Yeterli (yeşil):** Stok kritik eşik üzerinde
- **Kritik (sarı):** Eşiğe yaklaşmış, sipariş verilmeli
- **Tükenmiş (kırmızı):** Servis dışı bırakılmalı
- 'Yeni Ürün Ekle' butonu ile menüye yeni kalem eklenir; 'Düzenle' ve 'Sil' ile mevcut ürünler yönetilir

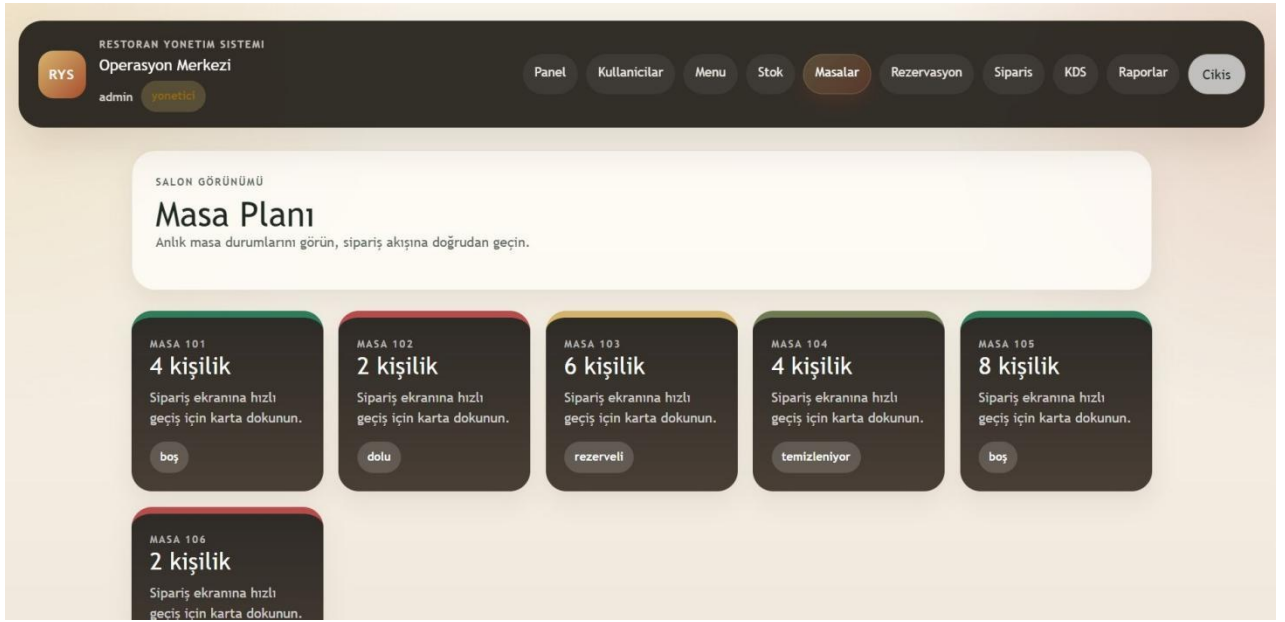


Şekil 7: Menü Yönetimi – ürün listesi, stok durumu ve işlemler

4.5 Stok Takip

Navbarda 'Stok' linkine tıklayın. Hammadde ve ürün stok seviyeleri izlenir.

- Her satırda: ürün adı, kategori, mevcut stok, kritik eşik ve durum
- 'Güncelle' alanına yeni stok miktarı girilerek anlık güncelleme yapılır
- 'Sadece kritik ürünler' filtresi ile acil takip gerektirenler ayrıştırılır



Şekil 8: Stok Takip — kritik seviye uyarıları ve güncelleme

4.6 Masa Planı

Navbarda 'Masalar' linkine tıklayın. Salon görünümünde masaların anlık durumu görselleştirilir.

Renk Kodu	Durum	Açıklama
Yeşil kenar	Boş	Müşteri yok, sipariş açılabilir
Kırmızı kenar	Dolu	Aktif müşteri var
Sarı kenar	Rezerveli	Ön rezervasyon mevcut
Koyu/Kahve	Temizleniyor	Müşteri ayrıldı, masa hazırlanıyor

Bir masa kartına tıklandığında o masanın seçili olduğu Sipariş Girişi ekranına yönlendirilirsiniz.

Şekil 9: Masa Planı — renkli durum kartları ve kapasite bilgisi

4.7 Rezervasyon Ekranı

Navbarda 'Rezervasyon' linkine tıklayın. Sol taraf rezervasyon formu, sağ taraf uygun masa özetidir.

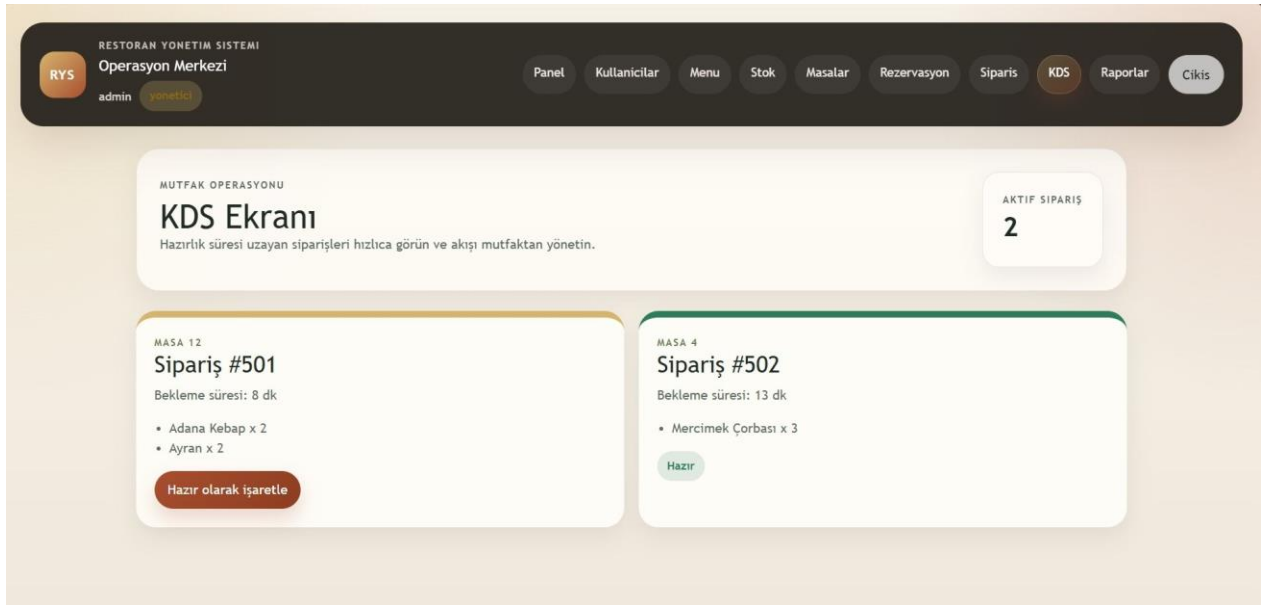
- Açılır listeden masa seçilir; sağ panelde o masanın anlık durumu görülür
- Müşteri adı, kişi sayısı ve rezervasyon tarihi/saati girilir
- 'Rezervasyon Oluştur' butonuyla masa durumu otomatik 'rezerveli' olarak güncellenir

Şekil 10: Rezervasyon Ekranı — form ve uygun masa özeti

4.8 Sipariş Girişi

Navbarda 'Sipariş' linkine veya Masa Planı'nda bir karta tıklayarak erişilir.

- Sol üstte kategori sekmeleri (Başlangıç, Çorbalar, Ana Yemek, İçecek, Tatlı)
- Seçili kategorideki ürünler listelenir; 'Sepete ekle' ile sipariş özetine eklenir
- Sağ taraftaki panelde ürün miktarları $+/ -$ ile düzenlenir
- 'Siparişi Onayla' butonu ile sipariş mutfağa iletilir ve KDS'de görünür hale gelir
- Backend bağlantısı yoksa uyarı mesajı gösterilir; sistem varsayılan listeye çalışır

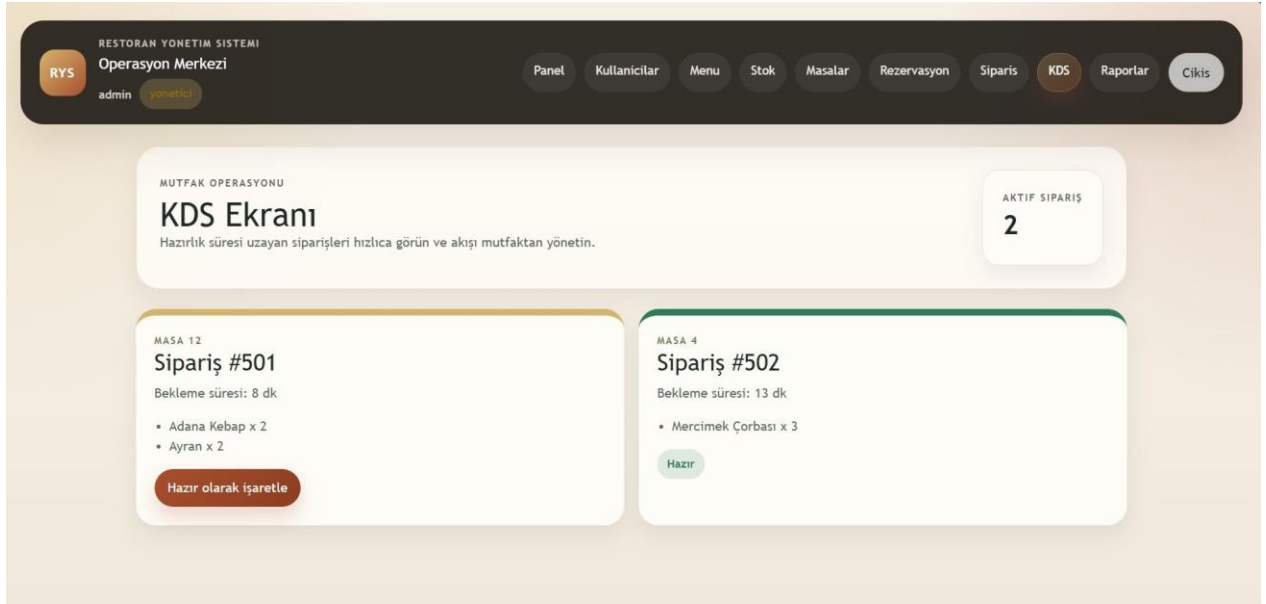


Şekil 11: Sipariş Girişi — kategori seçimi, ürün listesi ve sipariş özeti

4.9 KDS — Mutfak Görüntüleme Sistemi

Navbarda 'KDS' linkine tıklayın. Bu ekran mutfak personeli içindir.

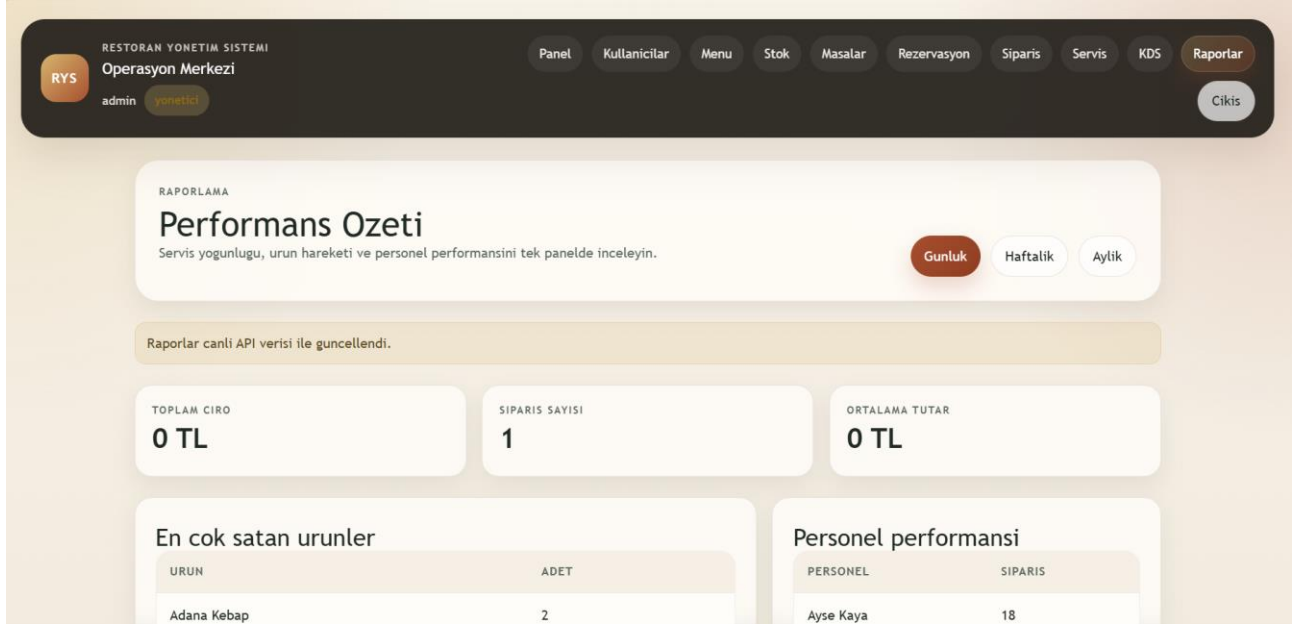
- Her aktif sipariş bir kart olarak gösterilir: masa no, sipariş no, bekleme süresi (dk), kalem listesi
- Hazırlandıktan sonra 'Hazır olarak işaretle' butonu aktif siparişi yeşil çerçeveye işaretler
- Sağ üstte aktif sipariş sayacı bulunur



Şekil 12: KDS Ekranı — aktif siparişler ve hazır işaretleme

4.10 Raporlar — Performans Özeti

Navbarda 'Raporlar' linkine tıklayın. Günlük / Haftalık / Aylık dönem seçilerek ciro, sipariş sayısı, ortalama tutar, en çok satan ürünler ve personel performansı incelenir.



Şekil 13: Raporlar – performans özeti, en çok satanlar ve personel analizi

5. Kurulum Talimatları

Bu bölüm projeyi sıfırdan kurmak isteyen kişi için adım adım rehber niteliğindedir.

5.1 Gereksinimler

Yazılım	Versiyon	Kontrol Komutu
Node.js	v18 veya üzeri	node --version
npm	v9 veya üzeri	npm --version
PostgreSQL	v14 veya üzeri	psql --version
Git	Herhangi	git --version

5.2 Repoyu Klonlama

```
git clone https://github.com/rmznfaruk/rys-bil204.git cd rys-bil204
```

5.3 Backend Kurulumu

Adım adım:

- **Dizine git:** cd backend
- **Bağımlılıkları yükle:** npm install
- **.env dosyasını oluştur:** cp .env.example .env
- **.env içeriği:**

```
PORT=5000 DATABASE_URL=postgres://postgres:sifreniz@localhost:5432/rys_db  
JWT_SECRET=gizli_anahtar_buraya JWT_EXPIRES_IN=7d
```

- Sunucuyu başlat:

```
npm start # → http://localhost:5000
```

5.4 Veritabanı Kurulumu

```
# Veritabanını oluştur psql -U postgres -c "CREATE DATABASE rys_db;" # Şemayı uygula  
psql -U postgres -d rys_db -f backend/db/schema.sql # Örnek verileri ekle (isteğe bağlı)  
psql -U postgres -d rys_db -f backend/db/seed.sql
```

5.5 Frontend Kurulumu

```
cd ../frontend npm install npm start # → http://localhost:3000
```

Önemli: Backend (port 5000) ve Frontend (port 3000) ikisi de aynı anda çalışır durumda olmalıdır. Backend başlatılmadan frontend'e gidildiğinde boş sayfa veya 'canlı veri alınamadı' uyarısı görünür — bu normal bir davranıştır.

5.6 İlk Giriş

seed.sql çalıştırıldıktan sonra varsayılan yönetici hesabı:

Kullanıcı Adı: admin Şifre: RysAdmin123!

6. Görev Dağılımı

Her ekip üyesinin dört proje aşamasındaki katkıları aşağıda ayrıntılı olarak verilmektedir.

Ramazan Faruk Acıöz

Aşama	Katkılar
Analiz Raporu	• SRS kontrolü, use-case düzenlemeleri, teknik gereksinim analizi
Tasarım Raporu	• Login sistemi mimarisi, yönetici paneli akışı, route yapısı
İmplementasyon	• server.js, authController.js, authMiddleware.js, LoginPage.jsx, Navbar.jsx, KorunanRota.jsx, YoneticiPaneli.jsx
Test ve Entegrasyon	• JWT testleri, rol kontrol testleri, PR merge ve entegrasyon yönetimi
Final Rapor	• Sistem entegrasyon özeti, ekip koordinasyonu, deployment kontrolü

İssa Abakar Mahamat

Aşama	Katkılar
Analiz Raporu	• Gerçek zamanlı sipariş akışı, rezervasyon gereksinimleri
Tasarım Raporu	• KDS ekran akışı, rezervasyon form tasarımı
İmplementasyon	• KDSEkrani.jsx, RezervasyonEkrani.jsx geliştirme
Test ve Entegrasyon	• Sipariş durum güncelleme testleri, API bağlantıları
Final Rapor	• KDS çalışma mantığı ve rezervasyon sistemi açıklamaları

Mehmet Selim Ceran

Aşama	Katkılar
-------	----------

Analiz Raporu	• Kullanıcı yönetimi ve raporlama ihtiyaç analizi
Tasarım Raporu	• Rapor ekranı grafik yapısı, kullanıcı CRUD tasarımı
İmplementasyon	• KullaniciYonetimi.jsx, RaporEkranı.jsx, routes/users.js
Test ve Entegrasyon	• CRUD testleri, rapor veri doğrulama işlemleri
Final Rapor	• Satış raporları ve kullanıcı yönetimi modülü açıklamaları

Hüseyin Kaya

Aşama	Katkılar
Analiz Raporu	• Masa yönetimi ve sipariş akış analizleri
Tasarım Raporu	• Masa planı UI tasarımı, sipariş akışı
İmplementasyon	• React kurulumu, MasaPlani.jsx, SiparisGirisi.jsx
Test ve Entegrasyon	• Sipariş API testleri, frontend-backend bağlantıları
Final Rapor	• Sipariş sistemi kullanım akışı ve UI geliştirme süreci

Muhammed Yusuf Uğur

Aşama	Katkılar
Analiz Raporu	• Veritabanı ilişkileri ve backend gereksinimleri
Tasarım Raporu	• API mimarisi, PostgreSQL şema planlaması
İmplementasyon	• schema.sql, pool.js, tables.js, orders.js, products.js, payments.js, reports.js, emailService.js
Test ve Entegrasyon	• Postman API testleri, transaction ve stok kontrol testleri

Final Rapor	• Backend servis mimarisi ve veritabanı açıklamaları
-------------	--

Serdar Kara

Aşama	Katkılar
Analiz Raporu	• Stok yönetimi ve ödeme sistemi gereksinimleri
Tasarım Raporu	• Menü yönetimi ve ödeme ekranı tasarımı
İmplementasyon	• MenuYonetimi.jsx, StokTakip.jsx, OdemeEkranı.jsx
Test ve Entegrasyon	• Ödeme işlemleri testleri, stok güncelleme kontrolleri
Final Rapor	• Ödeme sistemi ve stok takip modülü açıklamaları

7. Yaptıklarımız, Yapmadıklarımız ve Nedenleri

Bu bölümde analiz/tasarım raporlarında taahhüt ettiğimiz özellikler ile gerçekleştirip gerçekleştirilemediklerimiz dürüstçe aktarılmaktadır.

7.1 Tamamlanan Gereksinimler

Modül / Özellik	Durum	Notlar
Kullanıcı Girişi (JWT + BCrypt)	✓ Tamamlandı	Tüm roller çalışıyor
Yeni Kullanıcı Kaydı	✓ Tamamlandı	Garson, Kasiyer, Mutfak rolleri
Yönetim Paneli (Dashboard)	✓ Tamamlandı	Canlı özet metrikler
Kullanıcı Yönetimi	✓ Tamamlandı	Liste + ekleme
Menü Yönetimi (CRUD)	✓ Tamamlandı	Kategori, fiyat, stok
Stok Takip	✓ Tamamlandı	Kritik eşik uyarıları
Masa Planı (Salon Görünümü)	✓ Tamamlandı	Renk kodlu 4 durum
Rezervasyon Oluşturma	✓ Tamamlandı	Form + masa özeti
Sipariş Girişi	✓ Tamamlandı	Kategori / sepet / onay akışı
KDS Mutfak Ekranı	✓ Tamamlandı	Hazır işaretleme

Rol Tabanlı Erişim (RBAC)	✓ Tamamlandı	Middleware ile korunan rotalar
---------------------------	--------------	--------------------------------

7.2 Tamamlanamayan / Kısmen Tamamlanan Gereksinimler

Özellik	Durum	Neden Yapılamadı?
WebSocket / Gerçek Zamanlı Bildirimler	Yapılmadı	KDS için WebSocket altyapısı planlanmıştı. Node.js socket.io entegrasyonu zaman ve bilgi eksikliği nedeniyle hayata geçirilemedi.
Ödeme / Kasa Modülü	Yapılmadı	Adisyon kapatma ve ödeme tahsili akışı kapsam dışına alındı. Temel modülleri tamamlamak önceliklendirildi.
Kullanıcı Düzenleme (Rol Güncelleme)	Kısmen	Personel listeleme ve ekleme çalışıyor; mevcut kullanıcının rolünü değiştirme formu tamamlanamadı.
Mobil Responsive Tasarım	Kısmen	Temel responsive layout uygulandı; tablet ve telefon optimizasyonu yetersiz kaldı.

Özellik	Durum	Neden Yapılamadı?
E-Posta Bildirimleri	Yapılmadı	Analiz raporunda yer almıyordu; tasarım sırasında öngörüldü fakat önceliklendirilmedi.

7.3 Tasarımdan Sapmalar

- **Masa Durumu Geçişleri:** State diyagramında 'ödeme_bekleniyor' durumu planlanmıştı; ödeme modülü tamamlanamadığı için bu durum implementasyona yansıtılamadı.
- **Manager Sınıfları:** Tasarım raporunda OrderManager, TableManager gibi servis katmanı üstü sınıflar tanımlanmıştı. Express'in Controller–Service–Repository katman yapısına geçilince bu sınıflar doğrudan Service katmanına dahil edildi.
- **MySQL → PostgreSQL:** Başlangıçta MySQL düşünülmüştü; ücretsiz hosting ve ACID uyumluluğu nedeniyle PostgreSQL'e geçildi.
- **Canlı Sipariş API Entegrasyonu:** Sipariş sayfası backend bağlantısı kesildiğinde çöküyordu. Fallback varsayılan liste mekanizması sonradan eklendi.

7.4 Öğrenilen Dersler

- **Erken API Sözleşmesi:** Frontend ve backend ekipleri başlangıçta endpoint formatlarını netleştirmedikleri için entegrasyon aşamasında tekrar çalışma (rework) yaşandı. Bir sonraki projede OpenAPI/Swagger ile sözleşme önce yazılacak.
- **Zaman Yönetimi:** 'Zamanımız yetmedi' bir bahane değil, öğrenilmesi gereken bir gerçektir. MVP (Minimum Viable Product) kültürünü benimseyerek temel özellikleri bitirmek, tüm özellikleri yarım bırakmaktan değerlidir.
- **Test Önce:** Test yazma alışkanlığını geliştirme sürecine entegre etmek yerine sonraya bırakmak, hata düzeltme maliyetini artırdı.
- **Yapay Zekâ Okuryazarlığı:** AI önerilerini körce uygulamak yerine anlayarak kullanmak — kodu çalıştırmadan önce ne yaptığını bilmek — gerçek öğrenmeyi sağladı.